Class 6: R functions

Aarav Prasad (PID: A17440940)

Background

All functions in R have ar least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Lets call it `add()`

Input arguments can be either **"required"** or **"optional"**. The latter have fall-back default values that will be used if the user does not specify them.

> Q1a Your first version of the function should add two input numbers together. For example, add(4, 7) should return 11. 1 pt

```
add <- function(x, y = 0){
  x + y
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)). 1 pt

```
add <- function(x, y = 0){
  sum(x,y)
}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1] 21
```

Q1c Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) should return 2. Hint: explore the … (dots) argument or the base R function sum(). 2 pts

```
add <- function(...){
  sum(...)

}
```

```
add(1, 2, 13, 4)
```

```
[1] 20
```

```
ans <- add(1,2,3)
ans
```

```
[1] 6
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

```r
add <- function(x, y = 0, z = 0){
  return(sum(x,y,z))
  cat("Is it break time yet?\n")

}
add(10,100)
```

```
[1] 110
```

A generate_dna() function

A useful function here is the "base R" `sample()` function:

```r
sample(1:5, size = 60, replace = TRUE)
```

```
 [1] 2 3 2 4 3 5 5 3 4 1 1 2 2 3 1 4 4 4 1 1 4 3 4 2 3 3 5 2 5 5 3 3 2 4 2 2 4 2
[39] 4 2 4 2 4 5 5 1 3 1 2 3 3 3 4 4 1 1 3 5 4 5
```

We can use this to make a random nucleotide sequence if we draw from "A", "C", "G" and "T" ...

```r
sample(x = c("A", "C", "G", "T"), size = 10, replace = TRUE)
```

```
 [1] "C" "G" "G" "T" "G" "T" "C" "T" "G" "A"
```

> **Q2a**: Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A", "C". [1 pt]

```r
generate_dna <- function(len = 10){sample(x = c("A", "C", "G", "T"), size = len, replace = TR
generate_dna()
```

```
 [1] "T" "T" "G" "A" "A" "T" "C" "G" "A" "A"
```

> **Q2b**: Your second version should **optionally** be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a singe vector of multiple letters). For example "AAGGCTG". 1 pt

```
generate_dna <- function(len = 10, single.element = TRUE){ # len = sequence length
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)  # random bases
  if(single.element) {  # if want single string
    ans <- paste(ans, collapse = "") # join into one sequence

  }
return(ans)}  # output result
```

```
generate_dna(5, single.element = FALSE)
```

```
[1] "G" "G" "T" "T" "C"
```

```
generate_dna(5)
```

```
[1] "ACTGT"
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

> **Q2c**:Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length. For example: len9 CGAAGGCTG

```
cat("hello\nthere")
```

```
hello
there
```

```
generate_dna <- function(len = 10, single.element = TRUE){
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)

  ## Option to return single character string
   if(single.element) {
    ans <- paste(ans, collapse = "")

  }

## Format as FASTA with ID line
  cat( paste(">len", len, "\n", sep = ""))
  cat(ans)
```

```
  cat("\n")

}
```

```
generate_dna(5)
```

```
>len5
CATTC
```

Write a `generate_protein` function

> **Q3**: Write a function generate_protein() that returns a random peptide/protein sequence of a length specified by the user. For example generate_protein(6) might return "WQRTAG". Your function should: • Use the single-letter code for all 20 standard amino acids and no other letters (see earlier list at the beginning of this handout)and include clear comments that explain each step of your function. 2 pts # Example of the expected behavior generate_protein(6) # e.g. "WQRTAG"

```
## Making initial function, default length is 10, defaults to writing all in one (WTAG)
generate_protein <- function(len = 10, single.element = TRUE){
  ans <- sample(x = c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F",
"T", "W", "Y", "V"), size = len, replace = TRUE)

  ## Option to return single character string
   if(single.element) {
    ans <- paste(ans, collapse = "")

  }
return(ans)
  }
```

```
generate_protein(6)
```

```
[1] "VLRKWD"
```

Generate random protein sequences of length 6 to 13

Q4: Adapt and use your generate_protein() function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp. For example: id.6 NTYHEC id.7 MVRSIAW id.8 … Hint: the function cat() combined with paste() and the newline character `\n` makes this relatively straightforward. 2 pts

```
## Making initial function, default length is 10, defaults to writing all in one (WTAG)
generate_protein <- function(len = 10, single.element = TRUE){
  ans <- sample(x = c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F",
"T", "W", "Y", "V"), size = len, replace = TRUE)

  ## Option to return single character string
   if(single.element) {
    ans <- paste(ans, collapse = "")

   }

 ## Format as FASTA with ID line
  cat( paste(">id.", len, "\n", sep = ""))
  cat(ans)
  cat("\n")

}
```

```
for(l in 6:13){

  generate_protein(l)
}
```

```
>id.6
PCRCRM
>id.7
TVMQFET
>id.8
VSRSATRW
>id.9
GDLIYYLHW
```

```
>id.10
CPEKPPLQLS
>id.11
MLAMKYWVELH
>id.12
CYATHRTWVCDM
>id.13
DTMCKAKIRQMAQ
```

Are Our peptides "unique in nature"?

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Len	%id	%cov	unique
6	100	100	N
7	100	100	N
8	100	100	N
9	88	100	Y
10	90	100	Y
11	88	82	Y
12	100	75	Y
13	83	92	Y

Q5a: At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)? [1 pt] 9 nucleotides long is the first sequence that looks unique in nature.

Q5b: Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20 for a peptide of length) and to the size of the known protein universe. [1 pt] Short peptides match easily because the known protein database is huge relative to the small sequence space. As length increases, the possible sequences (20^L) grow rapidly, making exact matches unlikely.

Connecting our findings to immunology

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice

for the immune system to present very short peptides? [3 pt] Based on the data, the minimum length is around **9 amino acids**, since this is where peptides start appearing unique (no 100% identity and coverage). Very short peptides (6–8 aa) are found everywhere because the sequence space is small relative to the large protein database. Presenting such short peptides would risk the immune system recognizing self proteins as foreign. Longer peptides are more specific, reducing false activation and improving self vs. non-self discrimination.